

# **APLICACIONES WEB**

## **Conceptos Fundamentales**

**Nombre: Castillo Hernandez Maria Valentina**

**Grupo: G050401M**

## 1. Internet

**¿Qué es?** Es una red de redes a nivel mundial que interconecta millones de computadoras, servidores, celulares y dispositivos inteligentes mediante cables submarinos, fibra óptica, satélites y conexiones inalámbricas. Permite intercambiar información y soporta servicios como la Web, correo electrónico, videollamadas, redes sociales, streaming, almacenamiento en la nube y juegos en línea. Funciona con protocolos de comunicación como TCP/IP.

### Puntos clave

- Es una infraestructura global, no una sola computadora.
- Conecta distintos tipos de dispositivos y redes.
- Permite utilizar múltiples servicios, entre ellos la World Wide Web.
- Los protocolos de comunicación permiten que los equipos intercambien información.

### Ejemplo práctico

Cuando buscas una tarea en Google, ves un video en YouTube, mandas un mensaje o subes una fotografía, la información viaja mediante la infraestructura de Internet.

### Conclusión

Internet funciona como la infraestructura que conecta equipos y redes de todo el planeta y permite que múltiples servicios digitales estén disponibles.



## 2. Intranet

**¿Qué es?** Es una red privada que utiliza tecnologías y protocolos similares a los de Internet, pero cuyo acceso está limitado a usuarios autorizados dentro de una organización. Puede utilizar mecanismos de autenticación, contraseñas y firewalls para proteger la información interna.

### Puntos clave

- Está destinada a una organización o grupo específico.
- Su acceso es restringido.
- Puede centralizar información y servicios internos.
- Puede utilizar tecnologías web para facilitar el acceso.

### Ejemplo práctico

Una universidad puede utilizar una intranet para que alumnos consulten calificaciones, horarios, pagos y avisos internos.

### Conclusión

Una intranet permite aprovechar tecnologías de Internet en un entorno controlado, facilitando la comunicación y organización interna.



### 3. Extranet

**¿Qué es?** Es una extensión de una intranet que permite compartir de manera controlada una parte de la información o de los servicios con usuarios externos autorizados, como clientes, proveedores o socios.

#### Puntos clave

- El acceso es parcial y controlado.
- Está orientada a la colaboración con personas externas.
- Puede facilitar procesos entre una empresa y sus proveedores o clientes.

#### Ejemplo práctico

Un proveedor puede entrar a una plataforma autorizada para consultar el estado de sus pedidos sin tener acceso a toda la información interna de la empresa.

#### Conclusión

La extranet sirve como un punto de colaboración entre una organización y usuarios externos que necesitan determinados recursos.



## 4. Web

**¿Qué es?** La World Wide Web (WWW) es un servicio que funciona sobre Internet y reúne documentos y recursos enlazados mediante hipervínculos. Estos recursos pueden contener texto, imágenes, videos y elementos interactivos que se consultan principalmente mediante navegadores.

### Puntos clave

- La Web es un servicio que utiliza Internet.
- Los documentos y recursos pueden estar relacionados mediante enlaces.
- El navegador permite acceder y presentar esos recursos.

### Ejemplo práctico

Al abrir un sitio desde un navegador y desplazarte entre sus páginas mediante enlaces, estás utilizando la Web.

### Conclusión

Internet es la infraestructura de comunicación; la Web es uno de los servicios que funcionan sobre ella.



## 5. Página web

**¿Qué es?** Es un documento electrónico que puede contener texto, imágenes, videos, enlaces, botones y otros elementos. Normalmente se visualiza mediante un navegador y puede construirse con tecnologías como HTML, CSS y JavaScript.

### Puntos clave

- HTML estructura el contenido.
- CSS permite definir presentación y estilos.
- JavaScript agrega comportamiento e interacción.

### Ejemplo práctico

Una página informativa de una universidad puede mostrar texto, fotografías, enlaces a trámites y formularios.

### Conclusión

Una página web es un documento que el navegador interpreta para presentar información y, cuando corresponde, permitir interacción.



## 6. Aplicación web

**¿Qué es?** Es un tipo de software que se ejecuta dentro de un navegador y permite realizar acciones, procesar datos y trabajar de manera interactiva, además de mostrar información.

### Puntos clave

- Permite interacción con el usuario.
- Puede procesar y enviar información.
- Puede conectarse con servicios y bases de datos mediante un servidor.
- Ejemplos del material: Gmail, Google Drive y Trello.

### Ejemplo práctico

En una aplicación de correo, el usuario puede iniciar sesión, redactar mensajes, enviarlos, recibir información y administrar su cuenta desde el navegador.

### Conclusión

La diferencia esencial frente a una página meramente informativa es que una aplicación web permite realizar tareas y participar activamente en un sistema.



## 7. Navegador web

**¿Qué es?** Es el programa encargado de interpretar y mostrar páginas y aplicaciones web. Convierte los recursos recibidos del servidor en una interfaz que el usuario puede visualizar e interactuar.

### Puntos clave

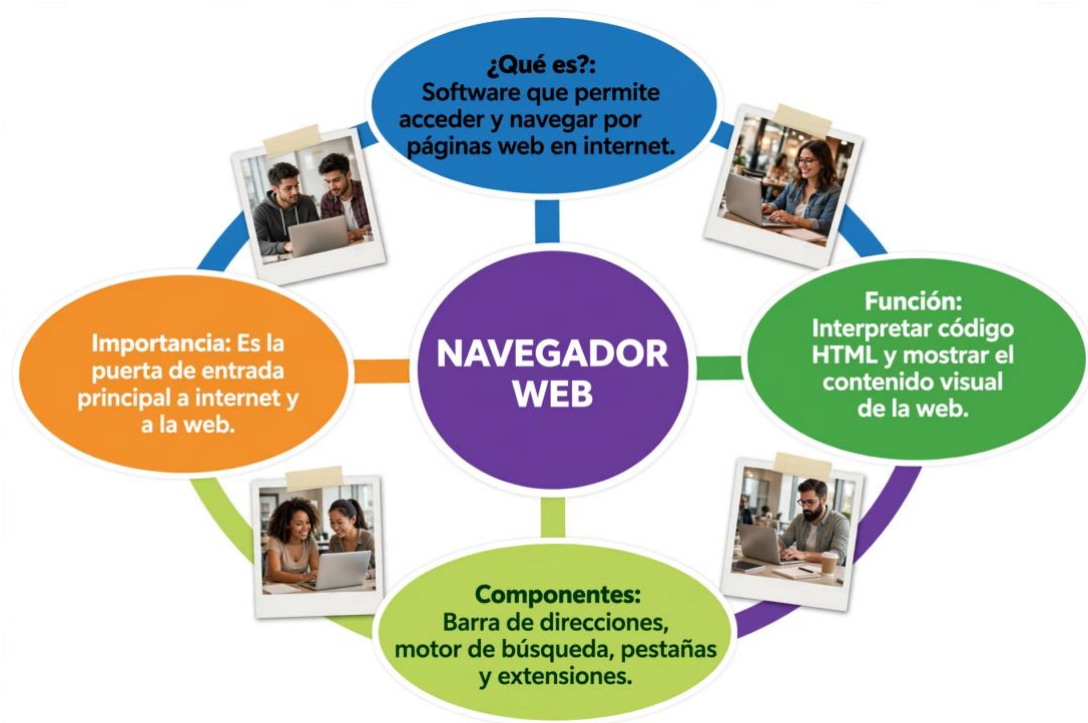
- Interpreta HTML, CSS y JavaScript.
- Permite navegar entre recursos mediante enlaces y direcciones.
- Ejemplos: Chrome, Firefox, Edge, Safari y Opera.

### Ejemplo práctico

Cuando escribes una dirección web en Chrome y presionas Enter, el navegador solicita los recursos necesarios y los presenta en pantalla.

### Conclusión

El navegador funciona como un puente entre el usuario y los recursos que ofrecen los sitios y aplicaciones web.



## 8. Front-end

**¿Qué es?** Es la parte visible de una página o aplicación web y comprende los elementos con los que el usuario interactúa directamente.

### Puntos clave

- Botones y enlaces.
- Menús de navegación.
- Formularios.
- Colores y tipografías.
- Animaciones e imágenes.
- Organización del contenido.

### Ejemplo práctico

En un formulario de inicio de sesión, el diseño de los campos, botones, textos y mensajes que ve el usuario pertenece al front-end.

### Conclusión

El front-end concentra la presentación y la interacción directa con el usuario, procurando que la interfaz sea clara y funcional.



## 9. Back-end

**¿Qué es?** Es la parte que funciona detrás de la interfaz visible. Se encarga de la lógica del sistema, procesamiento de información, seguridad y comunicación con bases de datos u otros servicios.

### Puntos clave

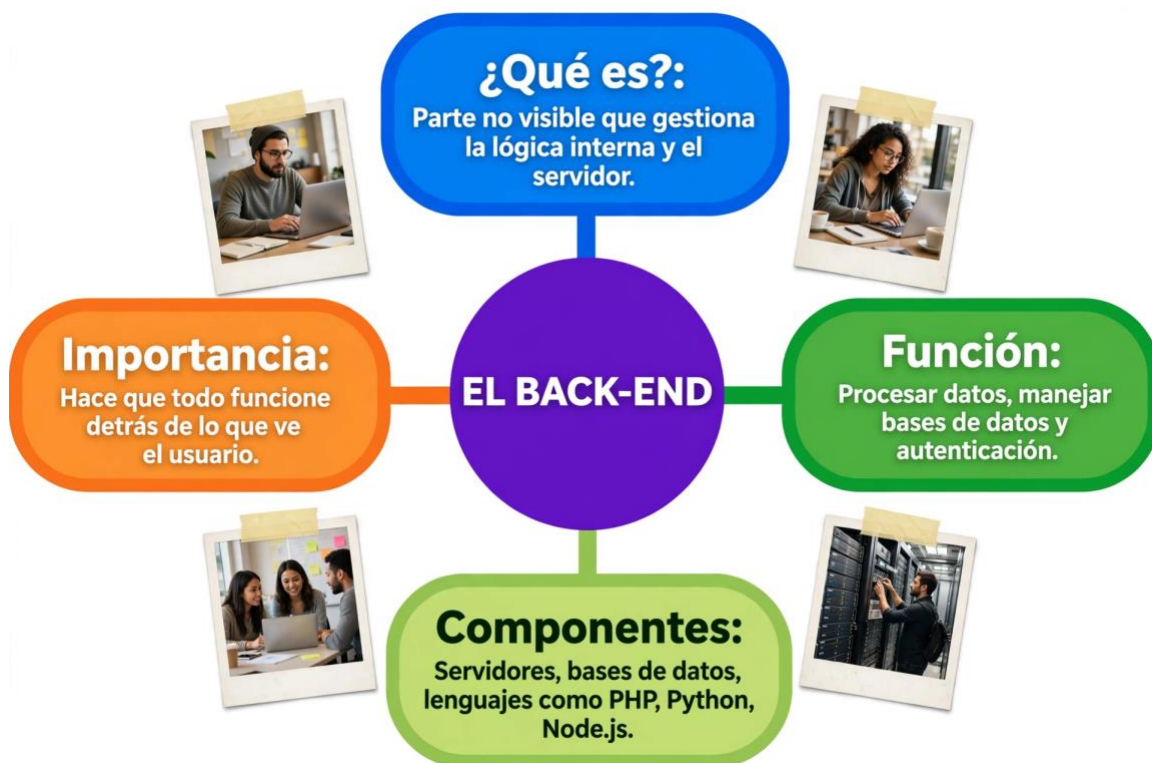
- Procesa solicitudes.
- Aplica reglas de negocio.
- Gestiona información.
- Puede comunicarse con bases de datos.
- Participa en procesos de autenticación y seguridad.

### Ejemplo práctico

Al iniciar sesión, el front-end presenta el formulario, mientras que el back-end recibe los datos y verifica si las credenciales son válidas.

### Conclusión

El back-end realiza gran parte del trabajo interno que permite que una aplicación web funcione más allá de lo que el usuario observa.



## 10. Hosting

**¿Qué es?** Es un servicio que proporciona espacio y recursos en un servidor para almacenar los archivos y componentes de un sitio o aplicación web, permitiendo que puedan estar disponibles a través de Internet.

### Puntos clave

- Los archivos se alojan en servidores.
- Permite publicar un proyecto para que pueda ser consultado en línea.
- El tipo de hosting puede variar según las necesidades del proyecto.
- Ejemplos mencionados en el material: Hostinger, Bluehost y Vercel.

### Ejemplo práctico

Después de desarrollar una página, el proyecto puede colocarse en un servicio de hosting para que otras personas puedan acceder a él mediante Internet.

### Conclusión

El hosting proporciona el lugar donde los recursos de un sitio o aplicación pueden permanecer disponibles para los usuarios.



## 11. HTTP

**¿Qué es?** HTTP (HyperText Transfer Protocol) es un protocolo de comunicación utilizado para intercambiar información entre un cliente, como un navegador, y un servidor web.

### Puntos clave

- Permite solicitudes y respuestas entre cliente y servidor.
- Se utiliza para obtener recursos web.
- Forma parte de la comunicación habitual de la Web.

### Ejemplo práctico

Cuando introduces una dirección web y el navegador solicita una página, se produce una comunicación mediante el protocolo correspondiente.

### Conclusión

HTTP establece una forma estandarizada para que clientes y servidores intercambien recursos e información.



## 12. HTTPS

**¿Qué es?** HTTPS (HyperText Transfer Protocol Secure) es la versión protegida de HTTP que utiliza mecanismos criptográficos basados en TLS para proteger la comunicación entre el navegador y el servidor.

### Puntos clave

- Protege la comunicación frente a terceros que intenten observarla o modificarla.
- Es especialmente importante cuando se manejan contraseñas, datos personales o pagos.
- El material resume la diferencia como HTTP sin cifrado de transporte y HTTPS con comunicación cifrada.

### Ejemplo práctico

En un sitio donde escribes una contraseña o datos de pago, HTTPS ayuda a proteger la información durante su transmisión.

### Conclusión

HTTPS aporta una capa de protección fundamental para la comunicación web y es especialmente importante para información sensible.



### 13. Modelo cliente-servidor

**¿Qué es?** Es una arquitectura de comunicación en la que un cliente inicia una petición y un servidor la recibe, procesa y devuelve una respuesta.

#### Puntos clave

- Cliente: inicia la petición.
- Servidor: procesa la solicitud y responde.
- El navegador suele actuar como cliente en aplicaciones web.
- La comunicación sigue un esquema de solicitud y respuesta.

#### Ejemplo práctico

Abres Firefox y solicitas una página. El navegador realiza la petición, el servidor procesa la solicitud y devuelve los recursos que serán mostrados.

#### Conclusión

El modelo cliente-servidor ayuda a entender cómo se distribuyen las responsabilidades entre quien solicita un recurso y quien lo proporciona.



## 14. Ruta absoluta

**¿Qué es?** Es una dirección que indica la ubicación completa de un recurso, incluyendo toda la información necesaria para localizarlo desde un origen determinado.

### Puntos clave

- Puede incluir un dominio completo.
- Puede utilizarse para identificar recursos externos.
- En sistemas locales puede incluir una ruta completa de carpetas.

### Ejemplo práctico

Una dirección como <https://www.ejemplo.com/assets/logo.png> identifica un recurso mediante una dirección completa.

### Conclusión

Una ruta absoluta proporciona una referencia completa y no depende de la carpeta actual para interpretar el recurso.



## 15. Ruta relativa

**¿Qué es?** Es una dirección que localiza un recurso tomando como referencia la carpeta o ubicación del archivo actual.

### Puntos clave

- Suele ser más corta que una ruta absoluta.
- Es útil para organizar recursos dentro de un mismo proyecto.
- Facilita mover el proyecto entre equipos o servidores cuando se conserva su estructura.

### Ejemplo práctico

Si un proyecto contiene index.html y una carpeta imagenes con logo.png, desde index.html puede utilizarse imagenes/logo.png para acceder al archivo.

### Conclusión

La ruta relativa resulta práctica en proyectos web porque relaciona los recursos con la estructura interna del proyecto.



## 16. DOM

**¿Qué es?** El DOM (Document Object Model) es una representación estructurada de un documento HTML en forma de árbol. Permite que JavaScript acceda y manipule los elementos de la página.

### Puntos clave

- Permite cambiar textos.
- Puede modificar estilos.
- Puede crear y eliminar elementos.
- Puede detectar clics y otros eventos.
- Conecta el código JavaScript con la estructura de la página.

### Ejemplo práctico

Al presionar un botón, JavaScript puede utilizar el DOM para cambiar un texto, mostrar un mensaje o modificar un elemento de la página.

### Conclusión

El DOM funciona como un puente entre JavaScript y la estructura del documento, haciendo posible modificar la interfaz mientras el usuario interactúa con ella.



## 17. Framework web

**¿Qué es?** Es un marco de trabajo que reúne librerías, componentes, herramientas y convenciones para facilitar y organizar el desarrollo de sitios y aplicaciones web.

### Puntos clave

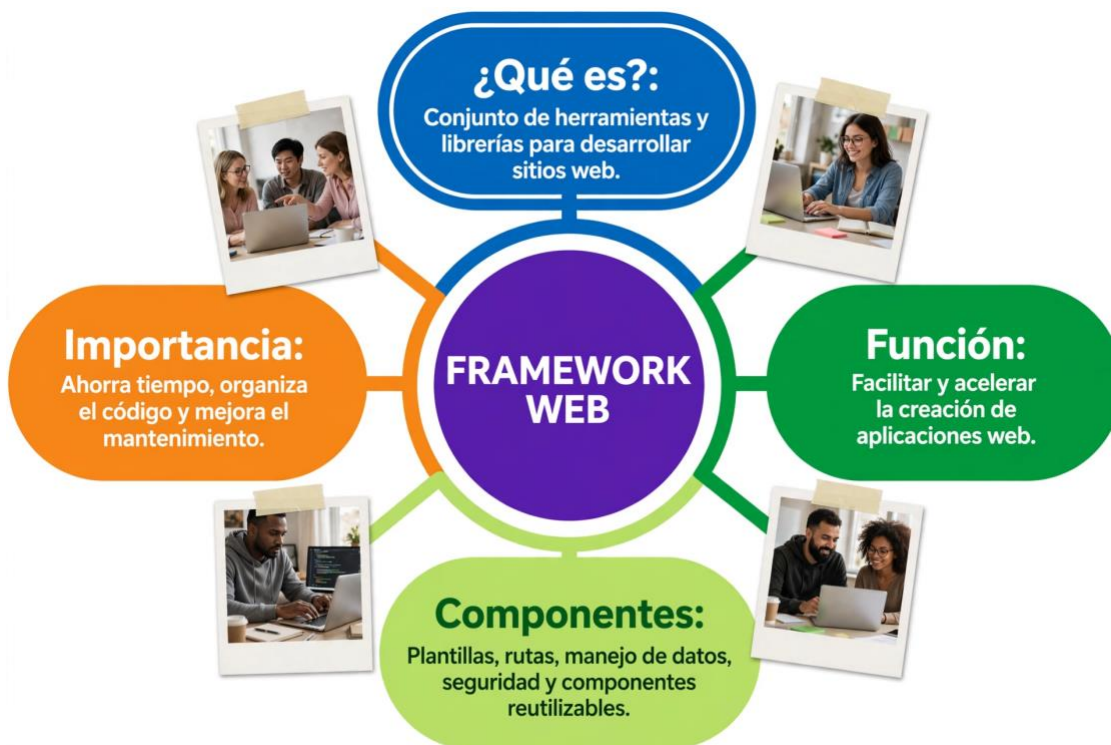
- Proporciona estructura para los proyectos.
- Evita repetir soluciones comunes.
- Incluye herramientas y componentes reutilizables.
- Puede ayudar a mantener un desarrollo más ordenado.

### Ejemplo práctico

En lugar de construir desde cero cada elemento de una interfaz, un desarrollador puede aprovechar componentes y utilidades proporcionados por un framework.

### Conclusión

Un framework proporciona una base de trabajo que ayuda a organizar el desarrollo y reutilizar soluciones ya construidas.



## 18. Bootstrap

**¿Qué es?** Bootstrap es un framework front-end de código abierto que ofrece componentes y utilidades predefinidas para construir interfaces de manera ágil y adaptable.

### Puntos clave

- Botones, formularios y alertas.
- Menús de navegación.
- Tarjetas y modales.
- Sistema de grilla de 12 columnas.
- Diseño responsive para diferentes tamaños de pantalla.

### Ejemplo práctico

Un desarrollador puede utilizar las clases de Bootstrap para crear rápidamente una tarjeta, un botón y una estructura adaptable sin diseñar cada componente desde cero.

### Conclusión

Bootstrap agiliza el desarrollo de interfaces al proporcionar componentes y un sistema de diseño reutilizable y adaptable.



## 19. Material Design

**¿Qué es?** Material Design es un sistema de diseño impulsado por Google que propone lineamientos, componentes y principios visuales para crear interfaces coherentes y usables.

### Puntos clave

- Botones y tarjetas.
- Barras y menús.
- Sombras y elevaciones.
- Tipografía y color.
- Animaciones y transiciones.
- Consistencia visual y de interacción.

### Ejemplo práctico

Una aplicación puede seguir principios de Material Design para mantener botones, tarjetas, menús, colores y jerarquías visuales coherentes entre sus pantallas.

### Conclusión

Material Design proporciona una guía visual y de interacción que ayuda a construir interfaces consistentes, ordenadas y centradas en la experiencia de uso.



## Referencias bibliográficas

1. **Mozilla Developer Network. (2025). Introducción a Internet. MDN Web Docs.**  
[https://developer.mozilla.org/es/docs/Learn/Common\\_questions/Web\\_mechanics/How\\_does\\_the\\_Internet\\_work](https://developer.mozilla.org/es/docs/Learn/Common_questions/Web_mechanics/How_does_the_Internet_work)
2. **IBM. (2024). ¿Qué es una intranet y una extranet? IBM Documentation.** <https://www.ibm.com/docs/es>
3. **W3C. (2024). The World Wide Web: Past, Present and Future.** <https://www.w3.org/>
4. **MDN Contributors. (2025). Qué es un navegador web y cómo funciona.**  
[https://developer.mozilla.org/es/docs/Learn/Common\\_questions/Web\\_mechanics/What\\_is\\_a\\_web\\_browser](https://developer.mozilla.org/es/docs/Learn/Common_questions/Web_mechanics/What_is_a_web_browser)
5. **Mozilla Developer Network. (2025). Cliente-Servidor: modelo de comunicación web.**  
[https://developer.mozilla.org/es/docs/Learn/Getting\\_started\\_with\\_the\\_web/How\\_the\\_Web\\_works](https://developer.mozilla.org/es/docs/Learn/Getting_started_with_the_web/How_the_Web_works)
6. **W3Schools. (2025). HTTP vs HTTPS - Diferencias y seguridad.**  
[https://www.w3schools.com/whatis/whatis\\_http.asp](https://www.w3schools.com/whatis/whatis_http.asp)
7. **MDN Contributors. (2025). Introducción al DOM.**  
[https://developer.mozilla.org/es/docs/Web/API/Document\\_Object\\_Model/Introduction](https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model/Introduction)
8. **GeeksforGeeks. (2025). Web Frameworks - Types and Benefits.** <https://www.geeksforgeeks.org/web-frameworks/>
9. **Bootstrap Team. (2025). Bootstrap 5.3 Documentation - Getting Started.**  
<https://getbootstrap.com/docs/5.3/getting-started/introduction/>
10. **Google. (2025). Material Design 3 - Foundations and Guidelines.** <https://m3.material.io/foundations>

## Conclusión general

---

El desarrollo web integra diferentes conceptos que cumplen funciones complementarias. Internet proporciona la infraestructura de comunicación; la Web organiza y distribuye recursos; el navegador permite acceder a ellos; el front-end presenta la interfaz; el back-end procesa la lógica y la información; los protocolos permiten la comunicación; el hosting mantiene disponibles los recursos; y herramientas como el DOM, los frameworks, Bootstrap y Material Design facilitan la construcción y manipulación de interfaces. Comprender estas relaciones permite tener una visión más clara de cómo una aplicación web pasa de ser un conjunto de archivos y servicios a una experiencia interactiva utilizada por una persona.